版本: v2.5 (2026-03-23 全书完成)

24.8 Vibe Coding 实践

> **本节目标**：深入解析 2025 Q3 兴起的新编程范式——Vibe Coding。这是 AI Native 时代人类与 AI 协作编写代码的革命性方法，代表从"语法驱动"到"意图驱动"的根本性转变。
>
> **核心来源**：Andrej Karpathy 技术演讲 (2025 Q3)、OpenAI 内部工程实践、GitHub Copilot Workspace 技术报告、Cursor 用户研究

24.8.1 Vibe Coding 定义与核心理念

定义

Vibe Coding 是一种 AI Native 编程范式，人类用自然语言描述意图 ("vibe")，AI 负责完整实现 (代码 + 测试 + 文档)，人类角色从"写代码"转变为"审代码"。

> **Andrej Karpathy 定义 (2025 Q3)**： "Vibe Coding 不是让 AI 帮你写代码，而是让 AI 写代码，你负责审核。人类从'实现者'变成'审查者'，从'语法专家'变成'意图专家'。"

范式转变：从语法驱动到意图驱动

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ 编程范式演进历程 │
├──────────────────────────────────────────────────────────────────────────────┤
│ │ │ 阶段 1 (1970s-2020s) 阶段 2 (2022-2023) 阶段 3 (2024+) │ │
┌──────────────┐ ┌──────────────┐ ┌──────────────┐│ │ │ Syntax- │ →
│ AI-Assisted│ → │ Vibe || │ │ Driven │ │ Coding │ │ Coding || │
└──────────────┘ └──────────────┘ └──────────────┘│ │ ↓ ↓ ↓ │ │ "手
写每一行代码" "AI 补全代码片段" "AI 实现完整功能" │ │ ↓ ↓ ↓ │ │ 人类 =
实现者 人类 = 主导者 人类 = 审查者 │ │ AI = 无 AI = 助手 AI = 实现者 │ │
│
└──────────────────────────────────────────────────────────────────────────────┘
```

三种范式对比:

| 维度 | Syntax-Driven | AI-Assisted | Vibe Coding |
|------|---------------|-------------|-------------| | **人类角色** | 实现者 (写代码) | 主导者 (改代码) | 审查者 (审代码) | | **AI 角色** | 无 | 助手 (补全/建议) | 实现者 (完整实现) | | **输入单位** | 语法语句 | 函数/模块 | 自然语言意图 | | **输出单位** | 单行代码 | 代码片段 | 完整功能 + 测试 + 文

档 | | **核心技能** | 语法熟练度 | Prompt 技巧 | 意图表达能力 | | **效率提升** | 1x | 2-3x | 10-100x |

人类角色转变：从"写代码"到"审代码"

传统编程中的人类职责： - 设计算法和数据结构 - 编写每一行代码 - 调试语法错误 - 编写测试用例 - 编写文档

Vibe Coding 中的人类职责： - 清晰描述意图 ("vibe") - 审查 AI 生成的代码质量 - 关注架构和安全 - 验收测试结果 - 决策是否合并

> **关键洞察**：人类不再需要精通语法细节，但需要更强的**系统思维**和**代码审查能力**。

AI 角色：实现者而非助手

AI-Assisted Coding 的局限： - AI 是"副驾驶"，人类仍是"驾驶员" - AI 只能补全人类已经开始的代码 - 人类仍需理解每一行代码的语法

Vibe Coding 的突破： - AI 是"实现者"，人类是"审查者" - AI 从 0 到 1 完整实现功能 - 人类关注"做什么"而非"怎么做"

```
┌──────────────
```

```
Step 1 │ → │ Step 2 │ → │ Step 3 │ → │ Step 4 │ │ │ │ 描述 │ │ AI
生成 │ │ Harness │ │ 人类 │ │ │ │ "Vibe" │ │ 实现 │ │ 验证 │ │ 审查
│ │ │ └──────────┘ └──────────┘ └──────────┘ └──────────┘ │ │ ↓ ↓
↓ ↓ │ │ 自然语言 代码 + 测试 Lint + 测试 架构 + 安全 │ │ 需求描述 + 文档
+ CI 最终验收 │ │ │ │ ↓ │ │ ┌──────────┐ │ │ │ 验证通过？ │ │ │
└──────┬───────┘ │ │ │ │ │ ┌──────────┴──────────┐ │ │ │
Yes │ │ No │ │ ▼ │ ▼ │ │ ┌──────────┐ │
┌──────────┐ │ │ │ 合并代码 │ │ │ 自修复循环 │ │ │ │ 完成任务
│ │ │ 失败→修复→验证 │ │ │ └──────────┘ │
└──────────┘ │ │ │ │ │ │ └──────────┘ │ │ │
└──────────────────────────────────────────────────────────────────┘
```

Step 1: 自然语言需求描述 ("Vibe")

什么是好的"Vibe"： - 清晰描述**目标**而非**实现方式** - 包含**约束条件**和**边界情况** - 提供**示例输入输出** - 说明**验收标准**

差的 Vibe vs 好的 Vibe:

| 差的 Vibe | 好的 Vibe | |---------|---------| | "写个登录功能" | "实现用户登录功能：支持邮箱/手机号 + 密码，密码需 bcrypt 加密，连续 5 次失败锁定 30 分钟，成功后返回 JWT token (有效期 24h)，需要单元测试覆盖所有边界情况" | | "优化这个函数" | "优化 `processData` 函数：当前处理 10 万条数据需要 5 秒，目标是 500ms 以内，可以改用并行处理或缓存，不能改变函数签名和返回值格式" | | "修复 bug" | "修复订单金额计算 bug：当商品有折扣且使用优惠券时，最终金额比预期多 10 元。已定位到 `calculateTotal` 函数第 47 行，折扣和优惠券的叠加逻辑有误" |

Step 2: AI 生成完整实现 (代码 + 测试 + 文档)

AI 输出标准:

```
┌──────────────────────────────────────────────────────────────────┐
│ AI 生成内容标准 │
├──────────────────────────────────────────────────────────────────┤
│ │ 1. 源代码 │ │ - 完整实现功能 │ │ - 遵循项目编码规范 │ │ - 包含必要
的注释 │ │ │ 2. 测试代码 │ │ - 单元测试覆盖核心逻辑 │ │ - 边界情况测试
│ │ - 错误处理测试 │ │ │ 3. 文档 │ │ - 函数/类说明 │ │ - 使用示例 │
│ - 注意事项 │ │ │ 4. 变更说明（如修改现有代码）│ │ - 修改了哪些文件 │
│ - 为什么修改 │ │ - 影响范围 │ │ │
└──────────────────────────────────────────────────────────────────┘
```

Step 3: Harness 自动验证 (Lint/测试/CI)

为什么需要 Harness: - AI 可能生成语法正确但逻辑错误的代码 - AI 可能忽略边界情况 - AI 可能违反项目规范 - Harness 提供自动化质量保障

验证流程:

```python

# Harness 验证流水线示例

class VibeCodingHarness: def validate(self, code_change): """验证 AI 生成的代码""" results = {}

```
    # 1. Lint 检查
    results['lint'] = self.run_linter(code_change.files)

    # 2. 类型检查
    results['type_check'] = self.run_type_checker(code_change.files)

    # 3. 单元测试
    results['unit_tests'] = self.run_tests(code_change.test_files)

    # 4. 集成测试
    results['integration_tests'] = self.run_integration_tests()

    # 5. 代码规范检查
    results['code_style'] = self.check_code_style(code_change.files)

    # 6. 安全检查
    results['security'] = self.run_security_scan(code_change.files)

    # 7. 性能回归测试
    results['performance'] = self.run_performance_tests()

    return results

def should_auto_fix(self, results):
    """判断是否自动修复"""
    # Lint 错误、格式问题 → 自动修复
    # 逻辑错误、测试失败 → 人工审查
    auto_fix_issues = ['lint', 'code_style']
    manual_review_issues = ['unit_tests', 'integration_tests', 'security']

    for issue_type in manual_review_issues:
        if not results[issue_type]['passed']:
            return False
```
```

```
    return True
```

Step 4: 自修复循环 (失败→修复→验证)

自修复策略:

```
┌─────────────────────────────────────────────────────────────────────┐
│ 自修复循环 │
├─────────────────────────────────────────────────────────────────────┤
│ │ 验证失败 │ │ ↓ │ │
┌──────────────────────────────────────────────────────────────────┐ │ │ │
分类错误类型： │ │ │ │ - Lint/格式问题 → 自动修复 (AI 重新生成) │ │ │ │
- 测试失败 → 自动修复 (AI 分析失败原因并修复) │ │ │ │ - 安全漏洞 → 人工审
查 (高风险) │ │ │ │ - 架构问题 → 人工审查 (需人类决策) │ │ │
└──────────────────────────────────────────────────────────────────┘ │ │ ↓
│ │ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 自动修复流程： │ │ │ │ 1. 将错误信息反馈给 AI │ │ │ │ 2. AI 分析错
误原因 │ │ │ │ 3. AI 生成修复方案 │ │ │ │ 4. 重新运行验证 │ │ │
└──────────────────────────────────────────────────────────────────┘ │ │ ↓
│ │ 验证通过？ → Yes → 完成 │ │ ↓ │ │ No (超过 3 次尝试) → 升级人工审查
│ │ │
└─────────────────────────────────────────────────────────────────────┘
```

自修复次数限制: - 同一问题自动修复最多 3 次 - 超过 3 次 → 升级人工审查 - 防止无限循环

Step 5: 人类最终审查 (关注架构和安全)

人类审查重点:

| 审查维度 | 审查内容 | 为什么人类审查 | |----------|----------|----------------| | **架构合理性** | 模块划分、依赖关系、扩展性 | AI 缺乏全局视角 | | **安全性** | 认证授权、数据加密、注入防护 | 安全风险需人类担责 | | **业务逻辑** | 是否符合业务需求、边界情况 | AI 不理解业务上下文 | | **性能影响** | 时间复杂度、资源消耗 | AI 可能忽略性能回归 | | **可维护性** | 代码可读性、注释质量 | AI 生成的注释可能不准确 |

人类可以忽略的: - 语法细节 (AI 已保证正确) - 格式规范 (Lint 已检查) - 基础测试覆盖 (Harness 已验证)

24.8.3 Vibe Coding 与 Harness Engineering 的关系

为什么 Vibe Coding 需要 Harness

核心论点：Vibe Coding 释放了 AI 的生产力，Harness Engineering 保障了输出的可靠性。两者缺一不可。

```
┌──────────────────────────────────────────────────────────────────────┐
│ Vibe Coding + Harness Engineering │
├──────────────────────────────────────────────────────────────────────┤
│ │ │ ┌──────────────┐ │ │ │ Vibe Coding │ │ │ │ (生产力) │ │ │
└──────┬──────┘ │ │ │ │ ▼ │ │ ┌──────────────────────────┐ │
│ │ AI 生成大量代码 │ │ │ │ (速度快、数量多) │ │ │
└──────┬──────────────┘ │ │ │ │ ▼ │ │ ┌──────────────┐ │
│ │ Harness │ │ │ │ (质量保障) │ │ │ └──────┬──────┘ │ │ │ │ ▼
│ │ ┌──────────────────────────┐ │ │ │ 可信任、可审计、可维护 │ │ │
│ 的高质量代码 │ │ │ └──────────────────────────┘ │ │ │ │ 没有
Harness 的 Vibe Coding = 失控的 AI │ │ 没有 Vibe Coding 的 Harness =
低效的人工 │ │ │
└──────────────────────────────────────────────────────────────────────┘
```

风险对比:

| 场景 | 风险 | 后果 | |------|------|------| | **Vibe Coding 无 Harness** | AI 生成错误代码无人发现 | 生产事故、安全漏洞 | | **Harness 无 Vibe Coding** | 人工编写代码效率低 | 无法发挥 AI 价值 | | **Vibe Coding + Harness** | 最小化风险，最大化效率 | 理想状态 |

上下文工程在 Vibe Coding 中的应用

Vibe Coding 中的上下文分层:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Vibe Coding Context Stack │
├──────────────────────────────────────────────────────────────────────┤
│ │ │ ┌──────────────────────────────────────────────────────────┐
│ │ │ Layer 1: 项目规范上下文 │ │ │ │ - 编码规范 (命名、结构、注释) │ │
│ │ - 技术栈约束 (语言版本、框架版本) │ │ │ │ - 架构模式 (MVC、微服务、事
件驱动) │ │ │ │ 作用：确保 AI 生成的代码符合项目风格 │ │ │
└──────────────────────────────────────────────────────────┘ │ │ ↑
│ │ ┌──────────────────────────────────────────────────────────┐ │
│ │ Layer 2: 领域知识上下文 │ │ │ │ - 业务术语和概念 │ │ │ │ - 现有代码
库结构 │ │ │ │ - API 接口定义 │ │ │ │ 作用：让 AI 理解业务含义，生成语义
正确的代码 │ │ │
```

```
└──────────────────────────────────────────────────────────────────────────────┘ │ │ ↑
│ │ ┌──────────────────────────────────────────────────────────────────────────┐ │
│ │ Layer 3: 任务约束上下文 │ │ │ │ - 性能要求 (响应时间、吞吐量) │ │ │
│ - 安全要求 (认证、加密、审计) │ │ │ │ - 兼容性要求 (浏览器、设备、API
版本) │ │ │ │ 作用：确保 AI 生成的代码满足具体约束 │ │ │
└──────────────────────────────────────────────────────────────────────────────┘ │ │ ↑
│ │ ┌──────────────────────────────────────────────────────────────────────────┐ │
│ │ Layer 4: 意图描述 ("Vibe") │ │ │ │ - 自然语言需求 │ │ │ │ - 示例
输入输出 │ │ │ │ - 验收标准 │ │ │ │ 作用：告诉 AI"做什么" │ │ │
└──────────────────────────────────────────────────────────────────────────────┘ │ │ │
└──────────────────────────────────────────────────────────────────────────────┘
```

上下文注入示例:

```python

# Vibe Coding 上下文构建

vibe_context = { # Layer 1: 项目规范 'project_rules': { 'language': 'Python 3.11+', 'style_guide': 'PEP 8 + Black 格式化', 'architecture': '领域驱动设计 (DDD)', 'testing': 'pytest + 90% 覆盖率要求', 'documentation': 'Google Style Docstring' },

```
# Layer 2: 领域知识
'domain_knowledge': {
    'business domain': '电商订单系统',
    'key_concepts': {
        'Order': '订单，包含多个 OrderItem',
        'OrderItem': '订单项，关联 Product 和数量',
        'Payment': '支付，支持多种支付方式'
    },
    'existing_modules': [
        'order_service.py - 订单核心逻辑',
        'payment_gateway.py - 支付网关集成',
        'inventory_service.py - 库存管理'
    ]
},

# Layer 3: 任务约束
'task_constraints': {
    'performance': '订单创建 API < 200ms (P95)',
    'security': '支付信息必须加密存储',
    'compatibility': '支持 MySQL 8.0+ 和 PostgreSQL 14+'
},

Layer 4: 意图描述
'vibe': '''
```
```

```
实现订单取消功能：

功能描述：
- 用户可以取消"待支付"状态的订单
- 取消后释放库存
- 如果已使用优惠券，返还优惠券
- 记录取消日志

输入：order_id, user_id, cancel_reason
输出：success/failure + 错误信息

边界情况：
- 订单不存在 → 返回错误
- 订单不是"待支付"状态 → 返回错误
- 用户不是订单所有者 → 返回错误
- 库存释放失败 → 回滚并报错

验收标准：
- 单元测试覆盖所有边界情况
- 取消操作原子性（要么全成功，要么全失败）
- 有完整的审计日志
'''

}```

**架构约束保障代码质量**

**架构约束类型**:

| 约束类型 | 示例 | 实现方式 | |----------|------|----------| | **依赖约束** | "不能直接访问数据库，必须通过 Repository" | 静态分析 + Lint 规则 | | **分层约束** | "Controller 不能调用其他 Controller" | 架构测试 (ArchUnit) | | **接口约束** | "所有外部调用必须有超时和重试" | 代码审查 + 静态分析 | | **安全约束** | "所有用户输入必须验证" | 安全扫描 + 代码审查 |

**架构约束注入示例**:

```yaml

# .harness/architecture_constraints.yaml

architecture_constraints: - name: "分层依赖约束" description: "确保代码遵循分层架构" rules: - "Controller 层只能调用 Service 层" - "Service 层只能调用 Repository 层" -
```
```

"Repository 层只能访问数据库" - "禁止跨层调用" enforcement: type: "static_analysis" tool: "archunit" fail_build: true

- name: "外部调用约束" description: "所有外部调用必须有保护措施" rules:
  - "HTTP 调用必须有超时 (默认 5s)"
  - "HTTP 调用必须有重试 (最多 3 次)"
  - "数据库查询必须有分页 (最大 1000 条)"
  - "所有外部调用必须有日志记录" enforcement: type: "code_review" checker: "external_call_checker.py" fail_build: true ```

熵管理清理 AI 生成冗余

问题: AI 倾向于生成冗余代码、过度注释、重复逻辑。

熵管理策略:

```
┌─────────────────────────────────────────────────────────────────────┐
│ 熵管理流程 │
├─────────────────────────────────────────────────────────────────────┤
│ │ │ AI 生成代码 │ │ ↓ │ │
┌─────────────────────────────────────────────────────────────────┐ │ │ │
熵检测： │ │ │ │ - 重复代码检测 (相同/相似代码块) │ │ │ │ - 冗余注释检测
(注释与代码语义重复) │ │ │ │ - 死代码检测 (未使用的函数/变量) │ │ │ │ -
过度复杂检测 (圈复杂度过高) │ │ │
└─────────────────────────────────────────────────────────────────┘ │ │ ↓
│ │ ┌─────────────────────────────────────────────────────────────────┐ │
│ │ 熵清理： │ │ │ │ - 自动提取重复代码为公共函数 │ │ │ │ - 删除冗余注释
│ │ │ │ - 删除死代码 │ │ │ │ - 重构过度复杂的函数 │ │ │
└─────────────────────────────────────────────────────────────────┘ │ │ ↓
│ │ 清理后的代码 (低熵状态) │ │ │
└─────────────────────────────────────────────────────────────────┘
```

熵检测工具:

```python class EntropyManager: def detect_entropy(self, code): """检测代码熵""" entropy_report = { 'duplicate_code': self.find_duplicates(code), 'redundant_comments': self.find_redundant_comments(code), 'dead_code': self.find_dead_code(code), 'high_complexity': self.find_high_complexity_functions(code) } return entropy_report

```
def clean_entropy(self, code, entropy_report):
    """清理代码熵"""
    cleaned_code = code

    # 1. 提取重复代码
```
```

```
    if entropy_report['duplicate_code']:
        cleaned_code = self.extract_common_functions(cleaned_code)

    # 2. 删除冗余注释
    if entropy_report['redundant_comments']:
        cleaned_code = self.remove_redundant_comments(cleaned_code)

    # 3. 删除死代码
    if entropy_report['dead_code']:
        cleaned_code = self.remove_dead_code(cleaned_code)

    # 4. 重构复杂函数 (需人工确认)
    if entropy_report['high_complexity']:
        self.flag_for_manual_refactor(entropy_report['high_complexity'])

    return cleaned_code
```

24.8.4 Vibe Coding 工具对比

主流 Vibe Coding 工具

| 工具 | 类型 | 核心特点 | 适用场景 | |------|------|----------|----------| | **Cursor** | AI 原生 IDE | 深度集成 AI，支持完整工作流 | 个人开发者、小团队 | | **GitHub Copilot Workspace** | 云端开发环境 | 与 GitHub 深度集成，支持任务规划 | 企业团队、开源项目 | | **Replit Agent** | 云端 IDE + Agent | 一键部署，适合快速原型 | 教育、快速原型 | | **OpenAI Codex** | API 服务 | 灵活集成，可自定义工作流 | 企业自研平台 |

Cursor (AI 原生 IDE)

优势： - 深度集成 AI，体验流畅 - 支持 Chat + Edit + Command 多种交互模式 - 本地运行，代码隐私性好 - 支持多文件编辑和重构

劣势： - 需要本地安装 - 团队协作功能较弱 - 自定义 Harness 能力有限

适用场景： - 个人开发者 - 小团队快速开发 - 对代码隐私有要求的场景

典型工作流： `1. 用自然语言描述需求 (Cmd+K) 2. AI 生成代码并自动应用 3. 人工审查 + 手动调整 4. Git 提交`

GitHub Copilot Workspace

优势： - 与 GitHub 深度集成 - 支持 Issue → Code 完整工作流 - 团队协作能力强 - 可配置 CI/CD 集成

劣势： - 需要云端运行 - 自定义能力有限 - 依赖 GitHub 生态

适用场景： - 企业团队 - 开源项目 - 已有 GitHub 工作流的团队

典型工作流： `1. 创建 GitHub Issue 描述需求 2. Copilot Workspace 分析 Issue 并生成计划 3. 人工确认计划 4. AI 生成代码并创建 PR 5. 人工审查 PR + CI 验证 6. 合并代码`

Replit Agent

优势： - 一键部署，零配置 - 适合快速原型 - 内置 hosting 和数据库 - 教育友好

劣势： - 锁定 Replit 平台 - 企业级功能不足 - 自定义能力弱

适用场景： - 快速原型验证 - 教学场景 - 个人项目

OpenAI Codex

优势： - 灵活集成，可自定义工作流 - 可构建专属 Harness - 适合企业自研平台

劣势： - 需要自行开发集成 - 成本较高 - 需要技术投入

适用场景： - 大型企业 - 需要定制化工作流 - 有技术团队支持

工具选择建议

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 工具选择决策树 │
├──────────────────────────────────────────────────────────────────────────┤
│ │ │ 你是个人开发者或小团队？ │ │ │ │ │ ├─ Yes → 需要本地运行/代码隐
私？ │ │ │ │ │ │ │ ├─ Yes → Cursor │ │ │ └─ No → 需要快速原型？ →
Replit Agent │ │ │ │ │ └─ No (企业团队) → 已有 GitHub 工作流？ │ │ │
│ │ ├─ Yes → GitHub Copilot Workspace │ │ └─ No → 需要高度定制？ →
OpenAI Codex │ │ │
└──────────────────────────────────────────────────────────────────────────┘
```

24.8.5 Vibe Coding 最佳实践

如何写好"Vibe"(意图描述技巧)

Vibe 描述模板：

```markdown
```markdown
```
```

功能描述

[用 1-2 句话描述功能目标]

输入输出

- 输入：[参数名 + 类型 + 说明]
- 输出：[返回值类型 + 说明]

功能要求

1. [要求 1]
2. [要求 2]
3. [要求 3]

边界情况

- [边界情况 1] → [预期行为]
- [边界情况 2] → [预期行为]
- [边界情况 3] → [预期行为]

约束条件

- 性能：[如 API 响应时间 < 200ms]
- 安全：[如 密码必须加密存储]
- 兼容：[如 支持 Python 3.9+]

验收标准

- [ ] 单元测试覆盖率 > 90%
- [ ] 所有边界情况有测试
- [ ] 通过 Lint 检查
- [ ] 文档完整

参考示例

[如有类似功能，提供参考代码或链接] ```

Vibe 描述示例:

```markdown
```

功能描述

实现用户注册功能，支持邮箱注册，发送验证邮件。

输入输出

- 输入：email (string), password (string), username (string)
- 输出：user_id (string) + 验证邮件发送状态

功能要求

1. 验证邮箱格式是否正确
2. 验证密码强度 (至少 8 位，包含大小写字母和数字)
3. 检查邮箱是否已注册
4. 密码使用 bcrypt 加密存储
5. 生成邮箱验证 token，有效期 24 小时
6. 发送验证邮件

边界情况

- 邮箱格式错误 → 返回 ValidationError
- 密码强度不足 → 返回 WeakPasswordError
- 邮箱已注册 → 返回 DuplicateEmailError
- 邮件发送失败 → 重试 3 次，仍失败则返回 EmailSendError

约束条件

- 性能：注册 API < 500ms (P95)
- 安全：密码 bcrypt 加密 (cost=12)，验证 token 使用 crypto.randomBytes
- 兼容：Python 3.9+, MySQL 8.0+

验收标准

- [ ] 单元测试覆盖率 > 90%
- [ ] 所有边界情况有测试
- [ ] 通过 Lint 检查 (Black + Flake8)
- [ ] 有 API 文档 (OpenAPI 格式)

参考示例

参考现有的 `login` 函数实现风格，保持代码一致性。```

代码审查要点 (关注什么、忽略什么)

人类应该关注的:

| 审查维度 | 审查要点 | 检查方法 | |----------|----------|----------| | **架构合理性** | 模块划分是否清晰、依赖是否合理 | 查看文件结构和 import 关系 | | **安全性** | 认证授权、数据加密、注入防护 | 重点审查敏感代码路径 | | **业务逻辑** | 是否符合需求、边界情况处理 | 对照 Vibe 描述逐条验证 | | **性能影响** | 是否有性能回归风险 | 查看算法复杂度和数据库查询 | | **可维护性** | 代码是否易读、命名是否清晰 | 快速浏览代码 |

人类可以忽略的:

| 可以忽略的 | 原因 | |----------|------| | 语法错误 | AI 生成 + Lint 已保证 | | 格式问题 | 自动格式化工具已处理 | | 基础测试 | Harness 已运行测试 | | 类型错误 | 类型检查器已验证 | | 简单注释 | AI 生成的注释通常准确 |

代码审查清单:

```markdown

# Vibe Coding 代码审查清单

## 架构审查

- [ ] 新代码是否符合项目分层架构
- [ ] 是否有循环依赖
- [ ] 是否有合适的抽象 (没有过度/不足)

## 安全审查

- [ ] 用户输入是否有验证
- [ ] 敏感数据是否加密
- [ ] 是否有 SQL 注入/XSS 风险
- [ ] 认证授权是否正确

## 业务逻辑审查

- [ ] 功能是否满足 Vibe 描述的所有要求
- [ ] 边界情况是否处理
```

- [ ] 错误处理是否合理

性能审查

- [ ] 是否有明显的性能问题 (N+1 查询、循环内 API 调用等)
- [ ] 是否有合适的缓存策略
- [ ] 数据库查询是否有索引

可维护性审查

- [ ] 代码是否易读
- [ ] 命名是否清晰
- [ ] 复杂逻辑是否有注释

测试审查

- [ ] 核心逻辑是否有单元测试
- [ ] 边界情况是否有测试
- [ ] 测试是否通过 ```

何时介入人工干预

必须人工干预的场景:

| 场景 | 原因 | 干预方式 | |------|------|----------| | **架构决策** | AI 缺乏全局视角 | 人类设计架构，AI 实现 | | **安全漏洞** | 安全风险需人类担责 | 人类审查安全相关代码 | | **业务逻辑复杂** | AI 不理解业务上下文 | 人类提供详细业务规则 | | **性能关键路径** | 性能优化需人类经验 | 人类指导优化方向 | | **自修复失败** | 超过 3 次自动修复仍失败 | 人类介入分析原因 |

可以不干预的场景:

| 场景 | 原因 | |------|------| | 简单的 CRUD 操作 | AI 能准确实现 | | 格式/样式调整 | Harness 自动处理 | | 测试代码生成 | AI 擅长生成测试 | | 文档编写 | AI 能准确描述代码 |

团队协作中的 Vibe Coding 流程

团队 Vibe Coding 流程:

```
┌──────────────────────────────────────────────────────────────────────┐
│ 团队 Vibe Coding 流程 │
├──────────────────────────────────────────────────────────────────────┤
│ │ 1. 需求分析 (人类) │ │ └→ 创建 Issue，描述需求 │ │ │ 2. Vibe 编写 (资深开发者) │ │ └→ 编写详细的 Vibe 描述 │ │ │ 3. AI 实现 (AI) │ │ └→ 生成代码 + 测试 + 文档 │ │ │ 4. Harness 验证 (自动化) │ │
```

```
└→ Lint + 测试 + CI │ │ │ │ 5. 代码审查 (资深开发者) │ │ └→ 架构 + 安全 + 业务逻辑审查 │ │ │ │ 6. 合并代码 (Tech Lead) │ │ └→ 最终审批 + 合并 │ │ │
└──────────────────────────────────────────────────────────────────────────────┘
```

角色分工:

| 角色 | 职责 | Vibe Coding 中的工作 | |------|------|---------------------| | **产品经理** | 需求定义 | 提供业务需求，不直接参与 Vibe Coding | | **资深开发者** | 架构设计 + 代码审查 | 编写 Vibe、审查 AI 输出 | | **初级开发者** | 学习 + 辅助审查 | 学习 Vibe Coding 方法，辅助审查 | | **Tech Lead** | 最终审批 | 审批关键代码合并 | | **AI** | 代码实现 | 生成代码、测试、文档 |

与 Git 工作流集成

Vibe Coding Git 工作流:

```bash

# 1. 创建功能分支

git checkout -b feature/vibe-user-registration

# 2. 编写 Vibe 描述 (保存为 .vibe 文件)

cat > .vibes/user-registration.vibe << 'EOF' [Vibe 描述内容] EOF

# 3. 运行 Vibe Coding (AI 实现)

vibe-code apply .vibes/user-registration.vibe

# 4. Harness 验证

harness run
```

5. 人工审查

(在 IDE 中审查 AI 生成的代码)

6. 提交代码

git add -A git commit -m "feat: 实现用户注册功能 (Vibe Coding)

- Vibe: .vibes/user-registration.vibe
- AI 生成代码 + 测试 + 文档
- Harness 验证通过
- 人工审查：架构/安全/业务逻辑已审查 "

7. 创建 PR

gh pr create --title "feat: 用户注册功能" --body "Vibe Coding 实现，详见 PR 描述"

8. 代码审查通过后合并

```

**PR 模板 (Vibe Coding)**:

```markdown

## Vibe Coding PR

### Vibe 描述

[链接到 .vibe 文件]

### AI 实现内容

- [ ] 源代码
- [ ] 单元测试
- [ ] 文档
- [ ] 变更说明

### Harness 验证结果

- [ ] Lint 检查通过
- [ ] 类型检查通过
- [ ] 单元测试通过
- [ ] 集成测试通过
- [ ] 安全检查通过

### 人工审查

- [ ] 架构合理性已审查
- [ ] 安全性已审查
- [ ] 业务逻辑已审查
- [ ] 性能影响已评估

### 自修复历史

- 自动修复次数：[X] 次
- 修复的问题：[列出]

### 注意事项

[需要 reviewer 特别关注的地方] ```

---

## 24.8.6 Vibe Coding 案例

**案例一：漫剧剧本生成项目的 Vibe Coding 实践**

**项目背景**： 漫剧剧本生成项目需要实现一个 AI 驱动的剧本创作系统，包括角色设定、情节生成、对话生成等模块。

**Vibe Coding 实践**:

**Vibe 1: 角色设定生成** ```markdown

# 功能描述

根据故事主题生成角色设定，包括姓名、年龄、性格、背景故事等。

# 输入输出

- 输入：story_theme (string), character_count (int, 3-10)

- 输出：List[Character] (角色列表)

## 功能要求

1. 角色姓名符合故事背景 (古代/现代/科幻)
2. 角色性格多样化，避免雷同
3. 每个角色有独特的背景故事
4. 角色之间有关系定义 (朋友/敌人/恋人等)

## 边界情况

- character_count < 3 → 返回 ValidationError
- character_count > 10 → 返回 ValidationError
- story_theme 为空 → 返回 ValidationError

## 约束条件

- 性能：生成时间 < 10 秒
- 质量：角色设定需通过人工审查标准

## 验收标准

- [ ] 单元测试覆盖所有边界情况
- [ ] 生成结果符合角色设定规范
- [ ] 有完整的日志记录 ```

**AI 生成结果**： ```python

# AI 生成的角色设定生成器

class CharacterGenerator: def **init**(self, story_theme: str): self.story_theme = story_theme self.name_generator = NameGenerator(story_theme) self.personality_pool = self._load_personality_pool()

```
def generate_characters(self, count: int) -> List[Character]:
    """生成角色列表"""
    if not 3 <= count <= 10:
        raise ValidationError(f"Character count must be between 3 and 10,

    if not self.story_theme:
        raise ValidationError("Story theme cannot be empty")
```

```
    characters = []
    relationships = self._generate_relationships(count)

    for i in range(count):
        character = Character(
            name=self.name generator.generate(),
            age=random.randint(18, 60),
            personality=self. select personality(i, count),
            background=self. generate background(i, self.story_theme),
            relationships=relationships[i]
        )
        characters.append(character)

    logger.info(f"Generated {count} characters for theme: {self.story_them
    return characters

def _generate_relationships(self, count: int) -> List[Dict]:
    """生成角色关系"""
    # AI 生成的关系逻辑...
    pass
```

Harness 验证结果: ✅ Lint 检查通过 ✅ 类型检查通过 ✅ 单元测试通过 (15/15) ✅ 集成测试通过 ⚠️ 性能测试：生成时间 8.5 秒 (接近上限 10 秒)

人工审查意见: ``` 架构审查: ✅ 符合项目分层架构 安全审查: ✅ 无安全问题 业务逻辑: ✅ 符合角色设定规范 性能审查: ⚠️ 生成时间接近上限，建议优化

审查结论：通过，建议后续优化性能 ```

Vibe 2: 情节生成 ```markdown

功能描述

根据角色设定和故事主题生成故事情节大纲。

输入输出

- 输入：characters (List[Character]), story_theme (string), act_count (int, 3-5)
- 输出：PlotOutline (包含各幕情节描述)

功能要求

1. 情节符合三幕/五幕结构
2. 每幕有关键事件和转折点

3. 情节与角色设定一致
4. 有清晰的冲突和解决

[... 详细描述 ...] ```

实践总结： - **效率提升**：原本需要 3 天开发的功能，Vibe Coding 仅需 4 小时 (含审查) - **代码质量**：Harness 保障了测试覆盖率和代码规范 - **人类专注点**：开发者专注于架构设计和业务逻辑审查，而非语法细节

案例二：OpenAI 内部实验案例 (3 人 100 万行代码)

案例背景： OpenAI 内部实验显示，3 人团队使用 Vibe Coding 方法，在 6 个月内生成了 100 万行代码。

关键数据： - 团队规模：3 人 - 时间周期：6 个月 - 代码量：100 万行 - 人均产出：约 5.5 万行/月 (传统开发约 3000-5000 行/月) - 效率提升：约 10-15 倍

成功因素： 1. **成熟的 Harness 体系**：自动化测试、Lint、CI/CD 完善 2. **清晰的 Vibe 描述规范**：团队成员都接受过 Vibe 编写培训 3. **严格的代码审查**：人类专注于架构和安全审查 4. **自修复机制**：80% 的问题自动修复，无需人工干预

挑战与解决： | 挑战 | 解决方案 | |------|----------| | AI 生成代码质量参差不齐 | 建立严格的 Harness 验证流程 | | 团队成员 Vibe 描述能力差异 | 建立 Vibe 模板和培训体系 | | 代码风格不一致 | 统一编码规范 + 自动格式化 | | 技术债务积累 | 定期熵管理清理 |

案例三：企业落地案例

某电商企业 Vibe Coding 落地:

背景： - 团队规模：50 人开发团队 - 业务场景：电商后台管理系统 - 痛点：需求变更频繁，开发效率低

落地过程:

阶段 1: 试点 (1 个月) - 选择 2 个小团队 (共 6 人) 试点 - 选择非核心功能模块 (报表生成、通知系统) - 建立基础 Harness 流程

阶段 2: 推广 (3 个月) - 扩展到 20 人团队 - 覆盖 50% 的新功能开发 - 完善 Vibe 描述规范和审查流程

阶段 3: 全面落地 (6 个月) - 全团队使用 Vibe Coding - 80% 的新功能通过 Vibe Coding 实现 - 建立企业级 Harness 平台

落地成果： | 指标 | 落地前 | 落地后 | 提升 | |------|--------|--------|------| | 需求交付周期 | 2 周 | 3 天 | 4.6x | | 代码审查时间 | 4 小时/PR | 1 小时/PR | 4x | | Bug 率 | 5% | 3% | 40%↓ | | 开发者满意度 | 60% | 85% | 42%↑ |

关键成功因素： 1. **高层支持**：CTO 亲自推动，提供资源支持 2. **渐进式落地**：从试点开始，逐步推广 3. **培训体系**：全员 Vibe Coding 培训 4. **工具支持**：自建企业级 Harness 平台

24.8.7 面试题目

面试题 1: Vibe Coding 基础理解

题目： 请解释 Vibe Coding 与传统 AI-Assisted Coding 的核心区别。在什么场景下应该使用 Vibe Coding，什么场景下应该使用传统开发方式?

考察点： - 对 Vibe Coding 概念的理解 - 对人类角色转变的理解 - 对适用场景的判断

参考答案要点： 1. **核心区别**： - AI-Assisted: AI 是助手，人类是主导者，人类写代码，AI 补全 - Vibe Coding: AI 是实现者，人类是审查者，人类描述意图，AI 实现

1. **人类角色**：
2. AI-Assisted: 人类仍需掌握语法细节

3. Vibe Coding: 人类关注意图表达和代码审查

4. **适用场景**：

5. Vibe Coding 适合：CRUD 功能、测试生成、文档编写、已知模式的实现
6. 传统开发适合：核心算法、性能关键路径、创新性问题、架构设计

面试题 2: Vibe 描述能力

题目： 请为以下功能编写 Vibe 描述："实现一个用户积分系统，用户完成特定任务获得积分，积分可以兑换商品或优惠券。"

考察点： - Vibe 描述的完整性 - 边界情况的考虑 - 约束条件的明确性

参考答案要点： ```markdown

功能描述

实现用户积分系统，支持积分获取、查询、兑换功能。

输入输出

- 输入：[根据具体 API 定义]
- 输出：[根据具体 API 定义]

功能要求

1. 积分获取：完成任务 (登录、下单、评价等) 获得积分
2. 积分查询：用户可查询积分余额和明细
3. 积分兑换：兑换商品或优惠券
4. 积分过期：积分有效期 1 年，过期自动清零

边界情况

- 积分不足时兑换 → 返回错误
- 并发兑换同一商品 → 防止超发
- 积分明细查询大数据量 → 分页处理
- 积分过期任务 → 每天定时执行

约束条件

- 性能：积分查询 API < 100ms
- 安全：积分变动必须有审计日志
- 一致性：积分变动必须事务性

验收标准

- [ ] 单元测试覆盖率 > 90%
- [ ] 并发测试通过
- [ ] 审计日志完整 ```

面试题 3: Harness Engineering 理解

题目： 为什么 Vibe Coding 必须配合 Harness Engineering？如果没有 Harness，Vibe Coding 会带来哪些风险？

考察点： - 对 Harness Engineering 价值的理解 - 对 Vibe Coding 风险的认识 - 对质量保障体系的理解

参考答案要点： 1. **为什么需要 Harness**： - AI 生成的代码可能有逻辑错误 - AI 可能忽略边界情况 - AI 可能违反项目规范 - Harness 提供自动化质量保障

1. **没有 Harness 的风险**：
2. 错误代码直接进入生产
3. 安全漏洞无人发现
4. 代码质量参差不齐

5. 技术债务快速积累

6. **Harness 核心组件**：

7. Lint/格式检查
8. 单元测试/集成测试
9. 安全检查
10. 性能测试
11. 代码审查流程

实战场景分析题 1

场景： 你是一个 10 人团队的 Tech Lead，团队准备引入 Vibe Coding。目前团队使用 GitHub + Jenkins 工作流，代码审查由资深开发者负责。

问题： 请设计一个 Vibe Coding 落地方案，包括： 1. 工具选择及理由 2. Harness 流程设计 3. 团队角色分工 4. 风险控制措施

考察点： - Vibe Coding 落地能力 - 团队管理能力 - 风险识别和控制能力

参考答案要点： 1. **工具选择**：GitHub Copilot Workspace (与现有 GitHub 工作流集成) 2. **Harness 流程**： - Lint: ESLint/Prettier - 测试：Jest + 覆盖率要求 90% - CI: Jenkins 流水线 - 审查：资深开发者审查架构和安全 3. **角色分工**： - 资深开发者：编写 Vibe、审查代码 - 初级开发者：辅助审查、学习 - Tech Lead: 最终审批 4. **风险控制**： - 试点开始 (2 人团队) - 非核心功能优先 - 严格审查流程 - 定期回顾和优化

实战场景分析题 2

场景： 团队使用 Vibe Coding 开发一个支付系统。AI 生成了支付处理代码，Harness 测试全部通过，但你在审查时发现一个潜在的安全问题：支付金额没有二次验证，可能存在篡改风险。

问题： 1. 这属于什么类型的问题？ 2. 你应该如何处理？ 3. 如何避免类似问题再次发生？

考察点： - 安全审查能力 - 问题处理能力 - 流程优化能力

参考答案要点： 1. **问题类型**：安全漏洞 (Harness 难以检测的业务逻辑安全问题) 2. **处理方式**： - 立即标记问题，不合并代码 - 向 AI 反馈安全问题，要求修复 - 修复后重新运行 Harness 验证 - 人工再次审查安全修复 3. **避免措施**： - 更新 Vibe 模板，增加安全约束要求 - 在 Harness 中增加安全检查项 - 建立支付系统安全审查清单 - 对团队进行安全培训

本章小结

Vibe Coding 代表了 AI Native 时代编程范式的根本性转变：

核心转变： - 人类从"写代码"变成"审代码" - AI 从"助手"变成"实现者" - 核心技能从"语法熟练度"变成"意图表达能力"

关键成功因素： 1. **高质量的 Vibe 描述**：清晰、完整、可验证的意图描述 2. **强大的 Harness 保障**：自动化验证确保代码质量 3. **有效的人类审查**：关注架构、安全、业务逻辑 4. **持续的熵管理**：清理 AI 生成的冗余代码

与 Harness Engineering 的关系： - Vibe Coding 释放生产力，Harness Engineering 保障可靠性 - 两者结合才能实现规模化、可持续的 AI 辅助开发

未来趋势： - Vibe Coding 将成为主流开发方式 - Harness Engineering 将成为企业标配 - 人类开发者角色将进一步向架构师和审查者转变

> **下节预告**：第 25 章将深入探讨 AI Agent 的性能优化与成本控制，包括 Token 优化、缓存策略、模型选择等实战技巧。